uIME 集成指南

uIME Integration Guide

uIME v0.1.0.51
2026-11-06
专有许可（二进制品个人用户免费）· Proprietary
UEFI Shell 中文输入法 · Chinese IME for the UEFI Shell

> 把汉字输入接进你自己的 UEFI 程序：**要交付的是哪个镜像、必须以什么顺序起来、宿主为什么一行都不用改、集成时必须绕开的那些坑，以及许可与发布的边界**。

目录 · Contents

（若下方目录为空，请在 Word 中按 F9 或右键选择“更新域”）

（目录域——在 Word 中更新后显示）

中文 | Chinese

这篇文档回答四件事：**要交付的是哪个镜像、必须以什么顺序起来、宿主程序为什么一行不用改、集成时要小心什么**。使用者视角的操作说明在 `docs/manual/index.html`。

01 · 交付物：一个产品

集成时最容易误解的就是镜像数量：**交付物只有一个**，其余都是构建过程的副产品。

交付物（客户只需要这一个）

文件	模块类型	大小	职责
`uime.efi`	**`DXE_DRIVER`**	~2.3 MB	**全部**。安装协议拦截层、建立悬浮窗、跑引擎，**常驻**。`load` 装的是它，编进固件卷的也是它

演示宿主（验证用，不属于产品本体）

文件	模块类型	大小	职责
`uimehost.efi`	`UEFI_APPLICATION`	~2.1 MB	演示宿主。用来证明汉字真的到了另一个程序里

产品只有 `uime.efi` 一个文件。 `uimehost.efi` 只是验证宿主，交付时不需要。

02 · 启动顺序（强制）

```
1）加载并常驻 uime.efi          ← 必须第一步
2）再启动需要中文的应用程序     ← 之后任意
```

顺序不是建议。拦截层必须已经就位，后续程序启动时读到的才是被包装过的协议实例。

生产环境：三种启动方式

```
Shell> fs0:                    # 先建立当前目录（或用绝对路径）
FS0:\> load uime.efi
Shell> YourApp.efi             # 之后任何读控制台的程序都能收到汉字

# 或：启动项指向 uime.efi（固件按 FilePath 加载）；或编进固件卷，磁盘上什么都不放
```

`load uime.efi` 现在可以直接用

`startup.nsh` 里用**绝对路径**（开机那一刻还没有当前目录）：

```
@echo -off
load fs0:\uime.efi
uimehost.efi
```

它有两个坑会让它失败，都与镜像类型、DEPEX、ConnectController 无关。

(1) 相对路径在开机时失败，那是 Shell 的正常行为。 Shell 启动后没有当前目录，`ls` 自己也报 `Current directory not specified.`。先 `fs0:`（或写绝对路径）即可，不是缺陷。

(2) 连绝对路径也可能失败——如果启动盘的 FAT 镜像是用自制工具生成的。 规范（fatgen103 §7.2.1）要求长名目录项的名字结束标志是 `0x0000`，`0xFFFF` 只能作余位填充；有生成器写成 `0xFFFF`。于是磁盘上每个带 LFN 的文件名尾部都多出一个 `U+FFFF`，`ls` 里肉眼可见：

```
FS0:\> ls
08/01/2026  12:00           2,366,432  uime.efi\uffff        <- 尾部的 U+FFFF
```

Shell 的按名查找于是整层失效：`load` / `type` / `ls <文件>` 都走 `ShellOpenFileMetaArg → EfiShellOpenFileList → EfiShellFindFiles → ShellSearchHandle`，最后用 `UnicodeCollation->MetaiMatch(名字，模式)` 逐个比较 ReadDir 返回的名字（`ShellProtocol.c`）。精确名字永远匹配不上 `uime.efi\uffff`，**只有 `*` 能匹配** —— 这正好解释了当时的怪现象：

```
FS0:\> ls                -> 正常列出（根节点路径，不做名字匹配）
FS0:\> ls fs0:\*         -> 正常列出（`*` 匹配任何字符串，带尾巴也行）
FS0:\> ls fs0:\uime.efi  -> ls: File Not Found
FS0:\> load uime.efi     -> load: File not found
FS0:\> uime.efi          -> 能运行（镜像派发）
```

为什么"按名直接打开"的那些路径不受影响：它们走 `FatSearchODir`（`FatPkg/EnhancedFatDxe/DirectoryManage.c`），先查长名哈希，失败后**若名字是合法 8.3 就回退到短名哈希**；`uime.efi` / `uime.cfg` / `uimehost.efi` 都是合法 8.3，所以照常命中 —— 这就是"有的路径能、有的不能"的全部原因。

一句话：`load` 只要求镜像类型是 `EfiBootServicesCode`（`UefiShellLevel2CommandsLib/Load.c`），`UEFI_DRIVER` 与 `DXE_DRIVER` 的 PE Subsystem 都是 `0x000B`，对 `load` 完全等价，不必改类型。`load -nc`（跳过 `ConnectAllEfi`）确实存在，但它在 `StartImage` **成功之后**才执行，不可能造成 `File not found`；而且驱动不装 DriverBinding，实测带不带 `-nc` 结果逐字节一致。

一个容易漏掉的细节：DeviceHandle 的传递

`LoadImage()` 用 SourceBuffer 方式加载的镜像**没有自己的 DeviceHandle**——它的 `LoadedImage->DeviceHandle` 是 `NULL`。而 `uime.efi` 需要知道自己所在卷，才能读写根目录的 `uime.cfg`。所以自己写的加载器必须在 `StartImage()` **之前**把 DeviceHandle 转交给驱动：

```
DrvLoaded->DeviceHandle = LoadedImage->DeviceHandle;
```

如果你自己写 loader，这一步不能漏。 漏了的表现是驱动能起来、能用，但设置永远写不进去，而且连一行错误都没有——配置文件只是静默地找不到。

部署形态

`load` 在所有形态下都能用，任选一种：

部署形态	怎么做	关键差别
Shell 里手动 `load uime.efi`	先 `fs0:` 建立当前目录，或写 `load fs0:\uime.efi`	交互式，改完立刻试
U 盘根目录放 `startup.nsh` 自动执行	用绝对路径：`load fs0:\uime.efi`	那一刻还没有当前目录，裸相对路径必然失败
启动项指向 `uime.efi`	固件按 FilePath 加载	连 Shell 都不需要：之后跑什么都带输入法
启动项指向 ESP 上的 `uime.efi`	固件用 FilePath 加载	DeviceHandle 就是分区，`uime.cfg` 照常读写
编译进固件卷（跟着 BIOS 一起刷）	见下	磁盘上什么都不用放，但要改一处代码

两条固件规则决定了能怎么启动，这不是 uIME 的设计选择，是镜像类型与内核行为：

规则	出处	后果
Shell 只启动 `ImageCodeType == EfiLoaderCode` 的镜像	`ShellPkg/Application/Shell/ShellProtocol.c`	`UEFI_DRIVER` 不能直接敲名字执行（"The image is not an application."）
DXE core 在 `StartImage` 返回时卸载一切 APPLICATION	`MdeModulePkg/Core/Dxe/Image/Image.c`	`UEFI_APPLICATION` 不能常驻，入口一返回就被释放
Shell 的 `load` 只接受驱动镜像	`UefiShellLevel2CommandsLib/Load.c`	APPLICATION 用 `load` 会被拒（not a driver）

三种模块类型对着两种启动方式（PE Subsystem 决定 `ImageCodeType`，它决定哪扇门开）：

INF 里的 `MODULE_TYPE`	PE Subsystem	直接敲名字	`load`	能否常驻
`UEFI_APPLICATION`	`0x000A`	可以	不行（not a driver）	不行，内核立刻卸载
`UEFI_DRIVER`	`0x000B`	不行（not an application）	可以	可以
`DXE_DRIVER`（本项目）	`0x000B`	不行（not an application）	可以	可以

为什么本项目是 `DXE_DRIVER`。 两个名字说的是约定：`UEFI_DRIVER` = 遵循 UEFI Driver Model（装 `EFI_DRIVER_BINDING_PROTOCOL`，由 `ConnectController` 找到设备后启动）；`DXE_DRIVER` = 在入口点把活干完的服务型驱动。本驱动没有设备要管、不装 DriverBinding，全部工作在入口完成，所以 `DXE_DRIVER` 才是如实的声明——edk2 里同类先例很多（`OvmfPkg/AmdSevDxe`、`IoMmuDxe` 都是 `DXE_DRIVER` 且零 DriverBinding；反过来抽样 40 个 `UEFI_DRIVER`，33 个装 DriverBinding）。**这不是行为差异**：BaseTools 为 `DXE_DRIVER` / `DXE_RUNTIME_DRIVER` / `UEFI_DRIVER` 生成同一个入口点（`BaseTools/Source/Python/AutoGen/GenC.py`），两者都产出 PE Subsystem `0x000B`，唯一差别是自动 include 哪个规范头。实测改类型前后 `load`、固件卷 dispatch、卸载自检、12 模式回归结果一致。**必须守住的是"别做成 APPLICATION"**：内核在 `StartImage` 返回时无条件释放 APPLICATION 镜像，而拦截层是原地改协议函数表、必须活过入口点。

固件内集成的两个坑，都出在"驱动不再从某个卷加载"这件事上：

1. **配置静默失效。** 固件卷 handle 上没有 `SimpleFileSystem`，`OpenConfigFile()` 拿到 `DeviceHandle` 后 `HandleProtocol()` 失败并返回 `EFI_NOT_FOUND`——读的时候被当成"第一次启动，还没有配置文件"，静默用默认值；保存时只留一行 `[Uime] settings not saved`。功能是好的，设置就是不持久化。要修就改 `UimeConfig.c` 的 `OpenConfigFile()`：拿不到卷时回退到枚举第一个带 `SimpleFileSystem` 的可写卷。

2. **dispatch 时机（实测踩到，已有解）。** 没有 `[Depex]` 的驱动会被**最早一轮**调度：`MdeModulePkg/Core/Dxe/Dispatcher/Dependency.c` 里写着，无 DEPEX 的驱动只要 `CoreAllEfiServicesAvailable()` 成立就直接可调度。而那一轮 GOP / `ConIn` 还不存在，入口装不出窗口 → 回滚 → 返回错误 → 镜像被卸载。第一次固件卷构建的现象很典型：FFS 确实在 `DXEFV` 里（`type=0x07`、2.37 MB），但 `dh -p gUimeInstalledProtocolGuid` 找不到 handle，宿主收到的是原始 ASCII。解法就是在驱动里写依赖：`[Depex] gEfiGraphicsOutputProtocolGuid AND gEfiSimpleTextInProtocolGuid`。DEPEX 只被 DXE dispatcher 检查，从文件加载（Shell 的 `load`、启动项、自己写的加载器）不评估它。

实测结论（2026-09-21）：加上这条 DEPEX 后，固件卷里的 `uime.efi` 由 dispatcher 自动加载并常驻——启动盘上**一个 uime 文件都不放**，宿主照样收到汉字；日志开头是 `[Uime] driver entry point reached (image code type 0x3)`（`0x3` 即驱动身份）。同一轮里的 `settings loaded (... fromfile=0)` 正好印证坑 1：同目录的 `uime.cfg` 写了 `candidates=9`，驱动仍从默认的 `cand=7` 起步。

自检标准（任何形态都适用）：驱动起来后看串口 `[Uime] settings loaded (cand=.. scheme=.. chinese=.. fromfile=1)`——`fromfile=1` 说明卷找到了、设置能持久化；`fromfile=0` 而你明明放了 `uime.cfg`，就是 DeviceHandle 不对。

拆解

```
Shell> dh                          # 列出 handle，找到驱动那个 image handle 的索引
Shell> unload <handle 索引>        # 索引从 load 的输出或 dh 列表里取
```

unload 收的是 handle 索引，不是文件名（Unload.c 用 ShellConvertStringToUint64 + ConvertHandleIndexToHandle，否则报 "Invalid handle"）——`unload uime.efi` 不是合法写法。先用 dh 找到那个 image handle 的索引。

卸载被设计成**失败时会拒绝**的操作：`Unload` 先恢复协议，再调 `UimeOverlayVerifyRestored()` 确认控制台真的回到原样。任何一步不过就返回错误码，镜像保持常驻。半途而废的卸载会让控制台继续指向一块即将被释放的镜像——那不是报错就完事的错误，是静默的内存损坏。

03 · 拦截是怎么做到的

三条到达路径，一个都不能漏

两个协议实例挂在 `gST->ConsoleInHandle` 上，其中一个还能通过系统表直达：

路径	谁在用
`gST->ConIn`	Shell，以及任何直接读系统表的程序
`HandleProtocol(ConsoleInHandle, SimpleTextIn)`	同一个实例，通常是这么取的
`HandleProtocol(ConsoleInHandle, SimpleTextInEx)`	LvglUefiPort 的 keypad indev

三者都必须最终连到输入法，否则输入法只对部分程序生效。

为什么是原地改函数表，而不是重装协议

教科书做法是 `ReinstallProtocolInterface()` 加 `gST->ConIn = &OurConIn`。**这个做法会把固件搞死**，实际观察过：Shell 在启动时会快照 `gST->ConIn`（`ShellParametersProtocol.c`），并在每条命令之后检查：

```
if (gST->ConIn != SystemTableInfo->ConIn) {
  CloseSimpleTextInOnFile (gST->ConIn);
  gST->ConIn = SystemTableInfo->ConIn;
}
```

`CloseSimpleTextInOnFile()` 不管拿到什么，都盲目地转成 Shell 自己的 `SHELL_EFI_SIMPLE_TEXT_INPUT_PROTOCOL` 然后 `FreePool()`。传进去一个**位于驱动镜像内**的协议结构体，池空闲链表立刻损坏，下一次分配会顺着一张垃圾链表走下去。

实测症状：加载驱动的那个应用再也退不出去，guest 在 DXE Core 里 100% CPU 空转，而且**没有任何一次调用进到 uIME 的代码里**。

所以 uIME **不动任何指针**。实例留在 `gST->ConIn` 和 handle 数据库原处，只覆盖它们的**函数表**，原表留着以便继续访问真实控制台。句柄和系统表数据库一行都没变，固件的簿记发现不了，而输入法站在了每一个消费者前面。

> 这是整个项目里最反直觉、也最不该被"改得更规范"的一处。如果你看到这行代码觉得「这样不干净，应该重装协议」——请先看上面那段 `FreePool`。

另外两个协议同理

包装	做法	要点
`PointerWrap`	原地改**所有** `AbsolutePointer` 实例	窗口内的点击要吃掉，不能透传给宿主，否则点按钮会同时命中宿主
`GopWrap`	原地改 `GraphicsOutput.Blt`	宿主 Blt 之后重新盖章窗口，**并且只重画 Blt 矩形与窗口的交集**

GOP 交集这一条是性能关键：宿主往往每帧整屏重绘，早期版本只要 Blt 与窗口有任何重叠就整窗口盖章，等于每步画两个窗口图像（实测 `draw=125k` 像素/步，而窗口本身只有 63k）。

没有定时器，泵点在宿主的按键调用上

`ReadKeyStroke(Ex)` 被宿主每帧调用，它同时充当输入法状态机的泵。定时器本来也行，但那会在 `TPL_NOTIFY` 上运行并可能在宿主渲染中途重入；借用宿主自己的调用点不会。

04 · 宿主为什么一行都不用改

宿主提交-call 到的永远是「一个按键」。拼音被引擎翻译成候选词，选中的汉字经**注入队列**变成一个 `UnicodeChar >= 0x80` 的按键，宿主读到它就像读到一个普通的 Unicode 按键。

```
用户敲 n-i-h-a-o
   → ConIn 包装层截获，进 IME 状态机
   → 拼音串 nihao + 候选「你好」
   → 用户按 1
   → 注入队列压入 U+4F60、U+597D 两个按键
   → 宿主调 ReadKeyStroke() 依次取到它们
```

宿主唯一要知道的是：**它可能收到 > 0x80 的字符**。任何能正确处理 Unicode 的宿主天然满足。

宿主必须满足的条件

条件	不满足会怎样
必须通过 `SimpleTextIn(Ex)` / `gST->ConIn` 读键	直接访问键盘控制器的程序完全绕过拦截层，一个汉字都收不到
必须轮询控制台	泵点就在宿主的读键调用上。宿主卡在模态等待里时，IME 窗口冻住——不刷新、不响应
不要用 `EfiBltVideoToBltBuffer` 回读自己那块屏幕	会读到被窗口盖住的像素。栈里没有回读通路，这点是明确的缺口

第一条和第二条是同一件事的两面：**既要用正确的接口，又要真的在用**。

05 · LVGL 应用特别注意：keypad indev 会抢键

`LvglUefiPort` 会在同一个 console handle 上装自己的 keypad indev，它**也**调 `ReadKeyStrokeEx`。两个读者一份队列，谁先跑谁赢。`uimehost.efi` 的处理方式值得照抄：

```
lv_indev_enable (Indev, false);   // 禁用，不是删除
```

必须禁用而不是删除：port 仍然持有这个 handle，会在 `LvglPortDeinit()` 里释放它。在这里删除会把关闭变成 double free。

另外顺序也重要——**每轮先把队列排空，再让 `LvglPortPoll` 泵下一帧**。这样应用才能拿到每一个键。对一个「要如实报告控制台送来了什么」的程序来说，被 indev 吞掉一个键看起来就像那个字符根本没到。

06 · 演示宿主怎么用

`uimehost.efi` 存在的全部意义是**证明汉字到了另一个程序**——不是自己画出来，而是从控制台收下来。它直接读 `SimpleTextInEx`（退化到 `gST->ConIn`），不走 LVGL keypad indev，所以它打印的每一个码元都确实是控制台送来的。

```
Shell> load fs0:\uime.efi
Shell> uimehost.efi
# 打字、选字；收到的汉字显示在上屏区
# ESC 退出，串口留下每个收到的码元
```

输出：

```
[UimeHost] READY
[UimeHost] recv U+4E2D
[UimeHost] recv U+56FD
[UimeHost] total=2 esc=1
```

自动化场景据此断言，不需要读一个像素。

07 · 集成检查清单

- `uime.efi` 已加载为常驻驱动——三种正规方式任选（见 §02）：Shell 里先 `fs0:` 再 `load uime.efi`（写进 `startup.nsh` 时用绝对路径 `load fs0:\uime.efi`）；启动项 `Boot####/Driver####` 指向它；编进固件卷由 dispatcher 自动加载（**必须带 `[Depex]`**，否则最早一轮被调度时 GOP/`ConIn` 还不存在）
- 知道"直接敲 `uime.efi` 不行"是正常的：Shell 派发只收 APPLICATION 镜像，驱动只能被 `load`、启动项或 dispatcher 启动——这是固件规则，不是本项目的选择
- 若自己写 loader：已把 DeviceHandle 转交给驱动（否则 `uime.cfg` 永远写不进去，且不报错）
- 需要中文的应用在 `uime.efi` 之后启动
- 应用通过 `gST->ConIn` / `SimpleTextIn` / `SimpleTextInEx` 读键，而不是直接访问键盘
- 应用轮询控制台（有键盘的地方要有 `ReadKeyStroke` 循环）

- 若用 LVGL：keypad indev 已 disable（不是 delete），且排泄队列在 pump 之前
- 卸载路径验证过（dh 取到 handle 索引后 `unload` 成功，串口有 `[Uime] unloaded`）

08 · 串口标记速查

串口是唯一的判据来源。以下标记按出现顺序：

来源	标记	含义
Uime	`[Uime] APP_VERSION=<ver>`	必须与 `expected_version.txt` 一致
UimeOverlay	`[UimeOverlay] install ok`	三个包装都装上了
UimeOverlay	`[UimeOverlay] selftest ok`	从外部验证过协议确实是自己的
Uime	`[Uime] window shown`	窗口已显示
Uime	`[Uime] installed, resident`	就绪，等宿主
Uime	`[Uime] settings loaded (... fromfile=1)`	`fromfile=1` 说明是从 `uime.cfg` 读的，不是默认值
Uime	`[Uime] hint=hotkeys` / `hint=select cand=N` / `hint=page P/N`	提示行三档状态
Uime	`[Uime] cand=N page=P pages=N pinyin=...`	候选、页码、当前拼音
UimeOverlay	`[UimeOverlay] deliver injected U+XXXX`	一个汉字注入出去了
UimeUi	`[UimeUi] window rebuilt (hint=... chars, settings=..., kbd=...)`	窗口重建后状态回填了多少
Uime	`[Uime] unloaded`	拆卸干净，可以卸载镜像

出错的标记：`selftest failed`、`unload refused`、`restore verification failed`、`inject queue full`、`pinyin buffer full`。其中 `[Uime] unload refused` 必须当故障处理——它意味着协议还是我们的，镜像必须保持常驻。

09 · 边界与已知问题

问题	现状
宿主不轮询控制台	窗口冻住。**设计如此**，没有定时器是刻意的（定时器会在 TPL_NOTIFY 重入宿主渲染）
宿主回读屏幕	会读到被窗口盖住的像素。缺口位置在代码注释里标明了
字库覆盖	GB2312 全量 6763 字；GBK / Unicode 扩展区的生僻字打不出来
窗口拖到屏幕边角之外	不会甩到对面——抓取位移按有符号运算，见产品手册 §09
卸载失败	不卸载，镜像保持常驻并报错。不要强行 `UnloadImage`
多实例	重复 `UimeOverlayInstall()` 返回 `EFI_ALREADY_STARTED`，不是错误

测试相关的注意点：`uime.cfg` 会持久化，**每个取证模式前重置磁盘镜像**。连续跑多个模式会让上一个模式写下的输入方案被下一个继承，`Ctrl+P` 循环到目标方案的次数就不对了——表现是某个模式突然失败，看起来像产品 bug，其实是测试隔离失效。

10 · 许可与发布

uIME 不开源。 源码不在公开渠道发布；对外提供的是 **`uime.efi` 二进制品**。

使用场景	是否需要授权
个人用户在自己（或自己拥有）的设备上安装使用	**免费**，无需申请
企业内部批量部署、随产品/硬件分发或预装、收费服务、外包集成	**需事先取得书面商业授权**

二进制品不得重新打包、改名、收费转售或声称拥有其权利；发行包应保持原样并保留 `LICENSE`。条款全文（中英双语，以中文为准）见仓库根目录的 `LICENSE`。

集成前请先确认自己的场景落在表格的哪一行——这一步比任何技术步骤都便宜。

第三方：拼音词库数据来自 `pinyin-data` / `phrase-pinyin-data` / `jieba`（均为 MIT，版权归各自作者）；五笔 86 码表来自 `limaofu/cof86`（MIT）。Rime 的 LGPL 词库被明确排除。`LvglPkg` 是独立项目（MIT），其随附字体与美术资源另有自身条款。

English | 英文

English

The above answers four things: **which image you ship, in what order it must start, why host applications need no changes, and what to watch out for.** For operator-facing usage see `docs/manual/index.html`.

01 · What ships

The most common misunderstanding is how many images there are: **one deliverable**, plus a by-product of the build.

Deliverables (a customer needs exactly one)

Image	Module type	Size	Role
`uime.efi`	`DXE_DRIVER`	~2.3 MB	**Everything.** Installs interception, owns the window, runs the engine, **stays resident**. This is what `load` starts and what goes into a firmware volume

Demonstration host (verification only, not part of the product)

Image	Module type	Size	Role
`uimehost.efi`	`UEFI_APPLICATION`	~2.1 MB	Demonstration host proving characters reach another program

The product is one file: `uime.efi`. `uimehost.efi` is only a verification host — leave it out of a shipment.

02 · Boot order (mandatory)

Production: three ways to start.

```
Shell> fs0:                  # install a cwd first (or spell the path absolutely)
FS0:\> load uime.efi
Shell> YourApp.efi           # anything started afterwards receives Chinese

# or: a boot option pointing at uime.efi (loaded by FilePath); or a firmware volume
```

The wrappers must already be in place for subsequently launched programs to see them.

`load uime.efi` works now

In `startup.nsh`, use an **absolute path** (there is no current directory that early):

```
@echo -off
load fs0:\uime.efi
uimehost.efi
```

There are two traps that make it fail - nothing to do with module type, DEPEX or ConnectController.

(1) **A relative path at boot fails, and that is normal Shell behaviour.** The Shell starts with no current directory; `ls` itself answers `ls: Current directory not specified.` Install one with `fs0:` (or spell the path absolutely) and it works.

(2) **Even absolute paths can fail - if the boot volume's FAT image was produced by a home-grown tool.** The spec (fatgen103 7.2.1) requires the long-name terminator to be **`0x0000`**, with `0xFFFF` reserved for padding; some generators write **`0xFFFF`**. Every long name then comes back with a trailing `U+FFFF`, visible in `ls`:

```
FS0:\> ls
08/01/2026  12:00           2,366,432  uime.efi\uffff        <- the trailing U+FFFF
```

That broke the Shell's whole name-lookup layer: `load` / `type` / `ls <file>` all go through `ShellOpenFileMetaArg -> EfiShellOpenFileList -> EfiShellFindFiles -> ShellSearchHandle`, which ends in `UnicodeCollation->MetaiMatch(name, pattern)` over the names ReadDir returned (`ShellProtocol.c`). An exact name can never match `uime.efi\uffff`; **only `*` can** - which explains every odd result seen then:

```
FS0:\> ls                -> lists fine (root node: no name matching)
FS0:\> ls fs0:\*         -> lists fine (`*` matches anything, junk included)
FS0:\> ls fs0:\uime.efi  -> ls: File Not Found
FS0:\> load uime.efi     -> load: File not found
FS0:\> uime.efi          -> runs (image dispatch)
```

Why the paths that open by name directly were never affected: they go through `FatSearchODir` (`FatPkg/EnhancedFatDxe/DirectoryManage.c`) - long-name hash first, and **if the name is a valid 8.3 name, the short-name hash as a fallback**. `uime.efi`, `uime.cfg` and `uimehost.efi` are all valid 8.3, so they were found. That is the entire reason some paths worked and others did not.

In one line: `load` only requires the image type to be `EfiBootServicesCode` (`UefiShellLevel2CommandsLib/Load.c`), and `UEFI_DRIVER` and `DXE_DRIVER` both produce PE Subsystem `0x000B` - they are equivalent to `load`, so there is no reason to change the type. `load -nc` (skip `ConnectAllEfi`) does exist, but it runs **after** a successful `StartImage` and cannot cause `File not found`; and since our driver installs no DriverBinding, measurements with and without `-nc` are byte-for-byte identical.

If you write your own loader, do not miss this: a SourceBuffer image has a NULL `DeviceHandle`, so `uime.efi` cannot find the volume its `uime.cfg` lives on. Hand your own handle over before `StartImage()`. Miss it and the driver works perfectly while settings silently never persist.

Deployment shapes

`load` works in every shape - pick whichever fits:

Deployment	How	Key difference
`load uime.efi` typed in a Shell	Install a current directory with `fs0:` first, or write `load fs0:\uime.efi`	Interactive; try it immediately
`startup.nsh` on a USB stick	Use an absolute path: `load fs0:\uime.efi`	No current directory exists yet, so a bare relative path always fails
Boot option pointing at `uime.efi`	The firmware loads by FilePath	No Shell at all: everything launched afterwards has the input method
Boot option pointing at `uime.efi` on the ESP	The firmware loads by FilePath	`DeviceHandle` is the partition and `uime.cfg` works normally
Compiled into the firmware volume	See below	Nothing on disk at all, but one code change is required

Two firmware rules decide what can be started how. Not design choices of uIME - image type and kernel behaviour:

Rule	Where it is written	Consequence
The Shell only starts `ImageCodeType == EfiLoaderCode`	`ShellPkg/Application/Shell/ShellProtocol.c`	A `UEFI_DRIVER` cannot be started by typing its name ("The image is not an application.")
The DXE core unloads every APPLICATION on `StartImage` return	`MdeModulePkg/Core/Dxe/Image/Image.c`	A `UEFI_APPLICATION` cannot stay resident; freed the moment its entry point returns
The Shell's `load` only accepts driver images	`UefiShellLevel2CommandsLib/Load.c`	An APPLICATION is refused with "not a driver"

The three module types against the two ways of starting an image:

`MODULE_TYPE`	PE Subsystem	Type it by name	`load`	Stay resident
`UEFI_APPLICATION`	`0x000A`	Yes	No (not a driver)	No, unloaded at once
`UEFI_DRIVER`	`0x000B`	No (not an application)	Yes	Yes
`DXE_DRIVER` (this project)	`0x000B`	No (not an application)	Yes	Yes

Why this project builds `DXE_DRIVER`. The two names describe a convention: `UEFI_DRIVER` means a driver that follows the UEFI Driver Model (installs `EFI_DRIVER_BINDING_PROTOCOL`, started by `ConnectController` once it finds a device); `DXE_DRIVER` means a service driver that does its work in the entry point. This driver has no device to manage and installs no DriverBinding, so `DXE_DRIVER` is the truthful declaration - edk2 is full of the same pattern (`OvmfPkg/AmdSevDxe`, `IoMmuDxe`: `DXE_DRIVER`, zero DriverBinding; conversely 33 of 40 sampled `UEFI_DRIVER` modules do install one). **It is not a behavioural difference:** BaseTools generates the same entry point for `DXE_DRIVER`, `DXE_RUNTIME_DRIVER` and `UEFI_DRIVER` (`BaseTools/Source/Python/AutoGen/GenC.py`), both produce PE Subsystem `0x000B`, and only the auto-included spec header differs. Measured: `load`, firmware-volume dispatch, the unload self-test and the 12-mode regression are identical before and after. **What must hold is "not an APPLICATION"**: the kernel frees an application image when `StartImage` returns, and the wrappers must outlive the entry point.

Two traps when the driver no longer comes from a volume. (1) *Settings go silently dead:* a firmware-volume handle has no `SimpleFileSystem`, so `OpenConfigFile()` fails and returns `EFI_NOT_FOUND` — on load that reads as "first boot, no file yet" and defaults are used without a word; on

save you get one line, `[Uime] settings not saved`. Fix it in `OpenConfigFile()` by falling back to the first writable volume exposing `SimpleFileSystem`. (2) *Dispatch order (hit in practice, solved):* a driver with no `[Depex]` is schedulable in the very first pass - `MdeModulePkg/Core/Dxe/Dispatcher/Dependency.c` says a missing dependency expression makes it an "UEFI 2.0 thing", dispatched as soon as `CoreAllEfiServicesAvailable()` holds - and in that pass GOP / `ConIn` do not exist, so the entry point cannot create the window, rolls back, and the image is unloaded. The first firmware-volume build showed it: the FFS is verifiably in `DXEFV` (`type=0x07`, 2.37 MB) yet `dh -p gUimeInstalledProtocolGuid` finds no handle and the host receives raw ASCII. The fix is one line in the driver: `[Depex] gEfiGraphicsOutputProtocolGuid AND gEfiSimpleTextInProtocolGuid`. A DEPEX is evaluated only by the DXE dispatcher; loading from a file (the Shell's `load`, a boot option, your own loader) never evaluates one.

Measured (2026-09-21): with that DEPEX, the `uime.efi` inside the firmware volume is dispatched automatically and stays resident — the boot volume carries **no uime file at all** and the host still receives Chinese; the log opens with `[Uime] driver entry point reached (image code type 0x3)` (`0x3` = driver identity). The same run logs `settings loaded (... fromfile=0)`, confirming trap 1: `uime.cfg` in that directory says `candidates=9` while the driver starts from the default `cand=7`.

Self-check for any shape: after the driver starts, look for `[Uime] settings loaded (cand=.. scheme=.. chinese=.. fromfile=1)` — `fromfile=1` means the volume was found and settings persist; `fromfile=0` while a `uime.cfg` demonstrably sits on the volume means the DeviceHandle is wrong.

Unload refuses rather than half-finishes: protocols are restored, then `UimeOverlayVerifyRestored()` confirms the console is genuinely back. A partial uninstall would leave the console pointing into an image about to be discarded — silent corruption, not an error code you can ignore.

03 · How interception works

Three consumer paths, all of which must reach the input method: `gST->ConIn`, `SimpleTextIn` on `ConsoleInHandle`, and `SimpleTextInEx` on the same handle (used by LvglUefiPort's keypad indev). Missing one makes the IME work for only some programs.

Why the function tables are patched in place rather than reinstalled: the textbook `ReinstallProtocolInterface()` plus `gST->ConIn = &OurConIn` deadlocks the firmware. The Shell snapshots `gST->ConIn` and after every command does:

```
if (gST->ConIn != SystemTableInfo->ConIn) {
  CloseSimpleTextInOnFile (gST->ConIn);
  gST->ConIn = SystemTableInfo->ConIn;
}
```

`CloseSimpleTextInOnFile()` blindly casts whatever it is handed and ends with `FreePool()` on it. Give it a structure living inside a driver image and the pool free list is corrupted. Observed: the loading application could never exit and the guest spun at 100% CPU inside DXE Core, with not one call reaching uIME code. Nothing in the handle or system table databases changes here, so no firmware bookkeeping notices — while every console consumer still goes through the input method.

> This is the least intuitive part of the project and the last thing that should be "cleaned up". If you are tempted to reinstall the protocol properly, re-read the `FreePool` above first.

`PointerWrap` patches **every** `AbsolutePointer` instance (clicks inside the window are swallowed, not passed to the host); `GopWrap` patches `GraphicsOutput.Blt` and re-stamps **only the intersection** of the blit rectangle and the window — a whole-window re-stamp cost two window images per step when the host repaints every frame.

There is **no timer**: `ReadKeyStroke(Ex)` is the pump, because a timer would run at `TPL_NOTIFY` and could re-enter the host mid-render.

04 · Why the host changes nothing

The host only ever asks "what key next?". Committed characters leave through an injection queue as `UnicodeChar >= 0x80` keys. The only thing a host must know is that it may receive characters above 0x80 — anything handling Unicode already does.

Requirement	Otherwise
Read keys through `SimpleTextIn(Ex)` / `gST->ConIn`	A program talking to the keyboard controller bypasses everything and gets no Chinese
Actually poll the console	The pump *is* the read call; a host blocked in a modal wait leaves the window frozen
Don’t read its own screen back	`EfiBltVideoToBltBuffer` returns pixels with the window composited in — a known gap

05 · LVGL applications: the keypad indev steals keys

`LvglUefiPort` installs a keypad indev on the same console handle, calling `ReadKeyStrokeEx` too. Two readers of one queue means whichever polls first wins. `uimehost.efi` handles it like this:

```
lv_indev_enable (Indev, false);   // disabled, not deleted
```

Disabled rather than deleted: the port still owns the handle and releases it in `LvglPortDeinit()`; deleting it here turns shutdown into a double free. Order also matters — **drain the queue at the top of each iteration, before `LvglPortPoll` pumps the next frame**.

06 · The demonstration host

`uimehost.efi` exists to prove characters arrive at another program, not merely that they can be drawn. It reads the console directly and prints one `[UimeHost] recv U+XXXX` line per received code unit, so a harness can assert without reading a pixel. Exits on ESC, reporting `[UimeHost] total=N esc=1`.

07 · Integration checklist

- `uime.efi` loaded as a resident driver - any one of three regular ways (see 02): `fs0:` then `load uime.efi` in a Shell (use the absolute path `load fs0:\uime.efi` in `startup.nsh`); a boot option (`Boot####` / `Driver####`) pointing at it; an FFS driver in a firmware volume, started by the dispatcher and **carrying `[Depex]`** (else it is dispatched in the first round, before GOP / `ConIn` exist)
- Understood that "typing `uime.efi` does nothing" is correct: Shell dispatch accepts only APPLICATION images, so a driver is started by `load`, a boot option or the dispatcher
- Own loader passes DeviceHandle to the driver (else `uime.cfg` never persists, silently)
- Chinese-capable applications start after `uime.efi`
- They read via `gST->ConIn` / `SimpleTextIn(Ex)`, not the keyboard controller
- They actually poll the console
- LVGL: keypad indev disabled (not deleted); queue drained before the pump
- Unload verified: `unload <handle index>` (from `dh`) succeeds and logs `[Uime] unloaded`

08 · Serial markers

Serial output is the source of truth. In order of appearance:

Source	Marker	Means
Uime	`[Uime] APP_VERSION=<ver>`	Must match `expected_version.txt`
UimeOverlay	`[UimeOverlay] install ok`	All three wrappers installed
UimeOverlay	`[UimeOverlay] selftest ok`	Verified from outside that the protocols really are ours
Uime	`[Uime] window shown`	Window is up
Uime	`[Uime] installed, resident`	Ready, waiting for a host
Uime	`[Uime] settings loaded (... fromfile=1)`	Loaded from `uime.cfg`, not defaults
Uime	`[Uime] hint=hotkeys` / `hint=select cand=N` / `hint=page P/N`	The three hint states
UimeOverlay	`[UimeOverlay] deliver injected U+XXXX`	A character went out
UimeUi	`[UimeUi] window rebuilt (hint=... chars, ...)`	How much state was restored after a rebuild
Uime	`[Uime] unloaded`	Clean teardown; safe to unload the image

Error markers: `selftest failed`, `unload refused`, `restore verification failed`, `inject queue full`, `pinyin buffer full`. Treat `unload refused` as a fault — protocols are still ours and the image must stay resident.

09 · Limits

Issue	Status
Host does not poll the console	Window freezes. **By design** — a timer would re-enter the host at TPL_NOTIFY
Host reads its own screen back	Gets pixels with the window composited in; gap marked in the source
Font coverage	Full GB2312 (6763 characters); GBK-only rarities cannot be typed
Unload failure	Refuses and keeps the image resident. Do not force `UnloadImage`
Double install	Returns `EFI_ALREADY_STARTED`; not an error

Testing note: `uime.cfg` persists, so **reset the disk image before each forensic mode**. Running modes back to back lets one mode's scheme leak into the next and changes where `Ctrl+P` lands — which looks exactly like a product bug and is really a loss of test isolation.

10 · Licensing and distribution

uIME is not open source. The source is not published; what is offered is the **`uime.efi` binary**.

Use	Licence needed
An individual installing it on a device they own	**Free**, no application needed
Bulk deployment inside a company, bundling with a product or hardware, pre-installation, paid services, outsourced integration	**A written commercial licence, obtained in advance**

Binaries may not be repackaged, renamed, resold for a fee or presented as your own; release packages must be passed on unmodified with `LICENSE` intact. The full terms (bilingual, Chinese prevailing) are in `LICENSE` at the repository root.

Settle which row your scenario falls in before integrating — it is cheaper than any technical step.

Third-party: dictionary data from `pinyin-data` / `phrase-pinyin-data` / `jieba` (all MIT, copyright their respective authors); the Wubi 86 code table comes from `limaofu/cof86` (MIT). Rime's LGPL dictionaries are deliberately excluded. `LvglPkg` is a separate project (MIT), and the fonts and artwork it ships carry their own terms.